

Blockchain-Based Ransomware-Resistant Data Sharing

System Design Document — Revised Architecture, Use Cases, and Semester Implementation Plan

1. Executive summary

This document presents a revised architecture for a blockchain-based, ransomware-resistant data sharing system. The original concept combined several strong ideas — content-addressed immutable storage, client-side encryption, an append-only smart contract ledger, entropy-based ransomware detection, proxy re-encryption for access control, and multi-signature governance. This revision keeps all of those ideas but resolves internal contradictions between them, closes gaps that would otherwise undermine the security guarantees, and separates what is realistic to build and demonstrate in a single semester from what should be documented as a design extension.

The system's core guarantee is narrow and specific: it does not prevent a workstation from being infected by ransomware, and it does not stop local files from being encrypted. What it guarantees is that a clean, verifiable copy of every file version an organization has previously shared through the platform remains retrievable and provably unaltered, even if every endpoint and every local server is compromised simultaneously.

Primary use case for this project

A consortium of independent, mutually distrusting organizations (for example, a hospital, a diagnostic lab, and an insurer) that need to share files with each other without any single member holding administrative control over the others' data. A single-organization deployment is a simplified variant of the same architecture, described in Section 4.6.

2. Threat model and scope

2.1 What the system defends against

- Ransomware that encrypts files on a local endpoint, file server, or mapped network drive and then attempts to overwrite or delete the shared copies of those files.
- An infected administrator account attempting to push a corrupted file into the shared system disguised as a legitimate update.
- A double-extortion attacker who exfiltrates data from the shared storage layer intending to leak it, where the storage layer itself never held plaintext.

- A single consortium member (or a single department within one organization) attempting to unilaterally alter or erase the historical record.
- A storage-node or validator-node operator attempting to tamper with a file without detection.

2.2 What the system does not defend against

- Initial infection of an endpoint. Endpoint protection (EDR, patching, user training) is a separate, complementary control and is out of scope.
- An attacker who compromises enough independent multisig signers to meet the governance threshold (see Section 4.5). This is a deliberate, documented trust assumption, not an oversight.
- Loss of a file that was never uploaded to the shared system in the first place — the platform protects shared organizational data, not purely local working files.
- Availability during a total network partition — retrieval requires reaching enough pinning nodes to reconstruct the file.

2.3 Two deployment variants — kept distinct

The original document blended a single-organization threat model (a rogue or compromised IT administrator inside one company) with a multi-organization consortium threat model (mutually distrusting independent entities). These require different governance shapes and are kept separate in this revision:

Variant	Trust boundary	Governance model	Primary risk addressed
Single organization	Departments within one company	Multisig across department heads (IT Security, Legal, Compliance, CTO)	A compromised IT admin account acting alone
Consortium (primary)	Independent legal entities	One validator/signer per member organization, threshold signing	One member unilaterally controlling shared data

3. Architecture overview

The system is organized into six layers. Each layer has a single responsibility and an explicit trust level, which is the main correction from the original design — several controls in the original document silently assumed a layer was trustworthy when it was not.

Layer	Responsibility	Trust level
1. Endpoint	Where the user works with files day to day	Untrusted — assume it can be fully compromised

Layer	Responsibility	Trust level
2. Client control	Client-side encryption, entropy-based anomaly flagging, upload agent	Semi-trusted — can be disabled by a sufficiently privileged attacker; treated as telemetry, not a hard boundary
3. Key custody	Storage and recovery of decryption keys and signing keys	Trusted, but deliberately kept off the endpoint
4. Storage	Content-addressed, redundantly pinned encrypted blobs (IPFS)	Untrusted for confidentiality (nodes never see plaintext); trusted for integrity via content hashing
5. Ledger (core)	Immutable, append-only record of file versions and CIDs	Trusted by construction — no owner, no update function
6. Governance	Multisig- and timelock-controlled parameters (validator set, ACLs)	Trusted only up to the signing threshold

The key architectural correction is that Layers 5 and 6 are separate smart contracts, not one contract wearing two hats. The original design asserted the ledger contract was fully ownerless and had no update methods, but separately described time-locked administrative actions against the same contract — those two claims cannot both be true of a single contract. Splitting them resolves the contradiction: the ledger core is genuinely immutable, and only the governance contract (controlling things like the validator list) is subject to multisig and timelock.

4. Component details

4.1 Endpoint layer (untrusted by design)

The endpoint is where ransomware first executes. No control that runs on the endpoint should be treated as a hard security boundary, because a sufficiently privileged attacker can disable it before it acts. This is an explicit gap in the original design, which relied on an endpoint-resident entropy daemon as if it were unbyassable. In this revision, the endpoint layer's controls are reclassified as defense-in-depth and early-warning telemetry, not as the source of the system's actual security guarantee — that guarantee comes from Layers 4 and 5, which the endpoint cannot reach or modify.

4.2 Client control layer — entropy detection and encryption

4.2.1 Entropy-based anomaly detection, corrected

The original design flagged any file with Shannon entropy near 7.9–8.0 bits/byte as likely ransomware-encrypted. This produces a high rate of false positives, because many legitimate file types are already high-entropy: compressed archives (.zip, .docx, .pptx, .xlsx are all zip containers), most image and video formats (.jpg, .mp4), and the system's own AES-256 ciphertext all sit in that same range. A absolute threshold cannot distinguish a newly-ransomed PDF from an ordinary photo.

Corrected approach: entropy is measured relative to a per-file, per-type historical baseline, not against a fixed global threshold.

- Maintain a rolling entropy baseline per file (or per file type, for new files) from prior clean versions recorded on the ledger.
- Flag a file only when its entropy jumps sharply relative to its own baseline (for example, a PDF moving from ~4.5 to ~7.9 bits/byte), not because it is simply high in absolute terms.
- Combine the entropy signal with behavioral indicators: mass file modification rate across the workstation, bulk extension changes, and creation of known ransom-note filenames. Entropy alone is a supporting signal, not a standalone verdict.

4.2.2 Client-side encryption

Every file is encrypted with AES-256 locally, before it leaves the endpoint, exactly as in the original design. This part of the design is sound: it means that anyone who gains access to the storage layer (Layer 4) — including its own operators — obtains only ciphertext. This property is unaffected by the corrections elsewhere in the design.

4.3 Key custody layer (new — closes a gap in the original design)

The original recovery workflow states that after wiping infected machines, the IT team decrypts the recovered file using “off-chain private keys.” The original document never specifies where those keys live between uploads, which is a critical omission: if the only copy of a decryption key was stored on the endpoint that just got wiped, the key is gone and the promised “zero-ransom recovery” is not actually possible.

This revision makes key custody an explicit layer with its own requirement: decryption keys and multisig signing keys must never reside solely on an endpoint that is a plausible ransomware target.

- Hardware security tokens (e.g., YubiKey, or an HSM for organizational keys) held by named individuals, separate from day-to-day workstations.
- A documented offline/cold backup of key material, following the same principle used for root certificate authorities.
- For the consortium variant, no single member's key alone should decrypt another member's files — this is what the proxy re-encryption layer (Section 4.7) is for.

4.4 Storage layer — content-addressed, redundantly pinned

IPFS content addressing is the right foundation and is unchanged from the original design: a file's identifier (CID) is a cryptographic hash of its contents, so an attacker cannot alter a file without changing its address, and any bit-level corruption or tampering is caught automatically when a requesting node re-hashes the retrieved bytes.

The gap in the original design is a persistence assumption. IPFS does not guarantee that content stays available indefinitely — a node only continues to serve content it is actively “pinning,” and unpinned content is eligible for garbage collection. If the only node pinning a clean version is the infected local machine, that version can vanish from the network even though its CID is still correctly recorded on the ledger. The fix is straightforward and is treated as a hard requirement, not an optimization:

- Deploy an IPFS Cluster (or equivalent pinning coordination) spanning multiple, independently operated nodes — one per consortium member at minimum.
- Every clean version is pinned redundantly at upload time, before the ledger entry is considered final.
- Retrieval should succeed as long as any one pinning replica is reachable.

4.5 Ledger layer — immutable core

The ledger core is a single-purpose smart contract: it appends {File ID, Version, CID, Timestamp, Author Signature, Parent CID} records and nothing else. It has no owner, no pause function, no delete function, and no upgrade proxy. This is what makes the earlier “ownerless contract” claim actually true, once the governance functionality described next is moved out of this contract entirely.

- Append-only: a compromised account can add a new (malicious) version, but cannot remove or alter any prior version.
- Self-verifying: because the CID is a content hash, the ledger entry itself cannot be forged to point to a different file without changing the CID.
- No administrative function exists to bypass, which is the property the original design wanted but had not actually achieved.

4.6 Governance layer — where administration actually happens

Anything that legitimately requires human judgment — adding or removing a validator, updating an access-control list, approving a system-wide parameter change — lives in a separate governance contract, not the ledger core. This is the resolution to the ownerless-contract/timelock contradiction described in Section 3.

Control	Mechanism	Effect
Multi-signature approval	M-of-N threshold signing (e.g., 3-of-5)	No single compromised account, including a compromised IT admin, can authorize a governance change alone

Control	Mechanism	Effect
Timelock	Mandatory delay (e.g., 72 hours) between proposal and execution	Gives monitoring systems and other signers a window to detect and cancel an unauthorized change
Scope limitation	Governance contract can only modify its own parameters (validator set, ACLs) — it has no function that can touch the ledger core	Even a fully compromised governance contract cannot rewrite history

Signer key hygiene matters as much as the threshold itself: if all five signers keep their keys on networked laptops, a single phishing campaign can plausibly compromise several at once. Signers should use hardware wallets, and the signer set should be diversified across departments or organizations, not job function alone.

4.7 Access control — proxy re-encryption (scoped for feasibility)

Proxy re-encryption (PRE) is the right long-term answer to “how do you share a file with another organization without a shared master key,” and the original design's description of it is directionally correct: a file is encrypted once, and re-encryption keys are generated per recipient without ever exposing plaintext to the storage layer. Full PRE with on-chain delegation and revocation is substantial cryptographic infrastructure, and building it from scratch is not realistic for a semester timeline.

Recommended scope: prototype PRE using an existing, audited library (for example, `umbral-pre`) against a handful of files to demonstrate the mechanism end to end, and document the full delegation/revocation design without implementing organization-scale key management. This keeps the core novel idea demonstrable while keeping the implementation surface bounded.

5. Points of confusion in the original design, and how they are resolved

#	Original claim	The contradiction / gap	Resolution in this revision
1	Contract is ownerless with no update methods, but also supports time-locked admin actions	A contract cannot be simultaneously unmodifiable and have an update path	Split into two contracts: an immutable ledger core, and a separate governance contract that is the only thing multisig/timelock controls
2	“Hash A remains pristine and untouched on the decentralized network”	IPFS only retains content that is actively pinned; an unpinned CID can be garbage-collected even though it is still recorded on-chain	Mandatory redundant pinning via an IPFS Cluster across independent nodes, required before a ledger entry is considered final

#	Original claim	The contradiction / gap	Resolution in this revision
3	Client-side entropy daemon detects ransomware before upload	The daemon runs on the same endpoint assumed to be compromised, so a privileged attacker can disable or bypass it	Reclassified as defense-in-depth / telemetry, not the system's security boundary; the real guarantee comes from the storage and ledger layers
4	Files near 7.9–8.0 bits/byte entropy are flagged as ransomware	Many legitimate file types (zip-based Office files, images, video, and the system's own ciphertext) are naturally in that range	Baseline-relative entropy comparison per file/type, combined with behavioral signals, not an absolute threshold
5	Recovery decrypts using “off-chain private keys” after wiping infected machines	Location of those keys during the incident is unspecified; if they lived only on the wiped machine, recovery is impossible	Explicit key-custody layer: hardware tokens or offline backup, never solely on an endpoint
6	Single-organization IT-admin threat model and multi-organization consortium threat model are described interchangeably	The two require different governance shapes (departmental multisig vs. cross-entity threshold signing)	Treated as two named variants (Section 2.3) with the consortium model as the primary target

6. Use cases

UC-1: Routine file upload and versioning

Actor: an employee at a consortium member organization, uploading a new or updated file through the client application.

1. The client control layer computes the file's entropy and compares it to the baseline for that file (or file type, if new).
2. If entropy is within expected bounds, the file is encrypted locally with AES-256.
3. The encrypted blob is pushed to the storage layer and pinned across the IPFS Cluster.
4. The resulting CID, author signature, timestamp, and parent CID (the previous version, if any) are appended to the ledger core.
5. The transaction is confirmed; the new version is now the current clean version of record.

UC-2: Sharing a file with another consortium member

Actor: Hospital A sharing a diagnostic scan with Specialist Lab B.

6. Hospital A generates a re-encryption key directed at Lab B's public key (proxy re-encryption).

7. The storage node transforms the ciphertext for Lab B without ever seeing the plaintext or either party's private key.
8. Before receiving the re-encryption key, Lab B signs an on-chain transaction acknowledging the access grant, creating a timestamped audit record.
9. Lab B decrypts locally using its own private key, held in its own key custody layer.

UC-3: Ransomware detection and isolation

Actor: an infected workstation attempting to sync a corrupted file.

10. The client control layer measures the file's entropy against its historical baseline and detects an anomalous jump, and/or the endpoint's behavioral monitoring flags mass file modification.
11. The client agent blocks the upload/pin request for the affected file and raises an alert.
12. The governance/monitoring layer is notified to flag the originating workstation for isolation by IT.
13. Because the upload was blocked before reaching the storage or ledger layers, no corrupted version enters the historical record.

UC-4: Recovery after a successful attack

Actor: IT team responding after ransomware has already encrypted local files (detection in UC-3 did not catch it in time, or the attack bypassed the endpoint agent entirely).

14. Infected local machines are wiped or reimaged, per standard incident response.
15. IT queries the ledger core for the last known-clean CID for each affected File ID, filtered to before the estimated infection timestamp.
16. The corresponding encrypted blob is retrieved from any available node in the IPFS Cluster.
17. The file is decrypted locally using the key retrieved from the key custody layer (hardware token or offline backup) — never from the wiped machine.
18. Recovery is complete without contacting the attacker or paying a ransom.

UC-5: Revoking a compromised endpoint's access

Actor: security team responding to a confirmed endpoint compromise.

19. Security submits a revocation request against the compromised workstation's delegation key.
20. The governance layer processes the revocation (this is a scoped, immediate action against a single delegation, distinct from the multisig/timelock path used for system-wide parameter changes).
21. The compromised endpoint's re-encryption key is invalidated; it can no longer request new re-encryption transformations.

22. Historical files remain unaffected — nothing needs to be re-uploaded or re-encrypted, since PRE delegations are per-recipient, not a shared master key.

UC-6: Governance change (e.g., adding a new consortium validator)

23. A proposal to add a new validator node is submitted to the governance contract.
24. The required threshold of signers (e.g., 3 of 5) approves the proposal.
25. The timelock delay begins (e.g., 72 hours), during which any signer can flag and cancel the change if it looks unauthorized.
26. If uncontested, the change executes automatically after the delay. The ledger core itself is never touched by this process.

UC-7: Insider leak and accountability

Actor: a partner organization suspects Lab B leaked a file it was authorized to access.

27. Investigators query the ledger for the on-chain access-grant signature associated with the file in question.
28. The signature is a non-repudiable, timestamped record that Lab B (specifically, the key that signed it) held access at that time.
29. Unlike a traditional server access log, this record cannot be edited after the fact by any administrator, including Lab B's own IT staff.

7. End-to-end workflow summary

The table below traces a single file from creation through a simulated ransomware incident and recovery, showing which layer is responsible at each step.

Stage	Layer(s) involved	What happens
Create	Endpoint → Client control	File authored locally; entropy baseline established
Encrypt	Client control → Key custody	AES-256 encryption using a key never persisted on the endpoint alone
Store	Storage (IPFS Cluster)	Encrypted blob pinned redundantly across independent nodes
Record	Ledger core	CID, signature, and timestamp appended; immutable from this point on
Share	Client control → Storage → Ledger	Proxy re-encryption key issued to recipient; access grant signed on-chain

Stage	Layer(s) involved	What happens
Attack	Endpoint (compromised)	Ransomware encrypts local copy; entropy/behavioral anomaly detected and upload blocked where possible
Contain	Governance	Compromised endpoint's delegation revoked; workstation isolated
Recover	Ledger core → Storage → Key custody	Last clean CID located, retrieved, and decrypted using safely-custodied key

8. Recommended technology stack for the semester build

8.1 Build fully

- Smart contracts: Solidity, developed and tested on Hardhat or Ganache (local testnet) — two contracts as described in Section 4.5–4.6: an immutable ledger core and a separate governance contract.
- Storage: a local IPFS deployment via Kubo, extended to an IPFS Cluster across 2–3 VMs or containers to demonstrate redundant pinning genuinely, not just a single node.
- Client agent: a Python or Node.js daemon performing AES-256 client-side encryption and baseline-relative entropy scoring.
- End-to-end demo: a scripted scenario — infect a VM, corrupt a tracked file, show detection and blocking, then show recovery of the clean version from the ledger and cluster.

8.2 Design and partially prototype

- Proxy re-encryption: prototype with an existing library (e.g., umbral-pre) against a small number of files to demonstrate the mechanism; document full delegation/revocation infrastructure as a design extension rather than building it from scratch.
- Multi-organization consortium: simulate with 2–3 local validator nodes rather than real separate institutions; the governance logic is the same regardless of how many physical organizations are represented.

9. Known limitations and future work

- Endpoint-resident controls (entropy daemon, encryption agent) can be disabled by an attacker with sufficient privilege on the endpoint; they are defense-in-depth, not the system's core guarantee.
- Availability depends on reachability of pinning nodes; a total network partition affecting all pinning replicas simultaneously would make retrieval temporarily impossible, though the ledger record itself would remain intact.

- Governance security is bounded by the signing threshold; compromise of M signers (not just one) defeats the multisig protection, which is why signer diversification and hardware-key custody matter.
- Full proxy re-encryption with on-chain, organization-scale key delegation and revocation is a substantial undertaking; this project treats it as a documented design with a small working prototype rather than a production implementation.
- On-chain storage cost was not modeled in detail; a production deployment would need gas/storage cost analysis appropriate to the chosen chain, which is why this project targets a local testnet rather than a public mainnet.

10. Glossary

Term	Definition
CID (Content Identifier)	A cryptographic hash of a file's contents, used as its address on IPFS. Changing the file changes the CID.
IPFS	InterPlanetary File System — a content-addressed, peer-to-peer storage network.
Pinning	The act of a node committing to continue storing and serving a given piece of content; unpinning content can be garbage-collected.
Proxy re-encryption (PRE)	A cryptographic scheme allowing a storage node to transform ciphertext for a new recipient without ever exposing the plaintext or either party's private key.
Multisig (M-of-N)	A transaction approval scheme requiring M signatures out of N designated keys before an action executes.
Timelock	A mandatory delay between a governance proposal's approval and its execution, allowing time to detect and cancel unauthorized changes.
Shannon entropy	A measure of randomness in data, in bits per byte; encrypted or compressed data tends toward high entropy, but is not uniquely identified by it.
Append-only ledger	A data structure that only permits adding new records, never modifying or deleting existing ones.